

Vesting Wallet Diff Audit



July 9, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Low Severity	7
L-01 In-Flight Deposits Can Block destroy_empty or Be Permanently Stranded	7
Notes & Additional Information	8
N-01 Incomplete sweep_settled Abort Documentation	8
N-02 Misleading Documentation	9
N-03 Open TODO in the Code	9
Conclusion	11
Appendix	12
Issue Classification	12

Summary

Type	Library	Total Issues	4 (1 resolved, 2 partially resolved)
Timeline	From 2026-07-06 To 2026-07-07	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	1 (1 resolved)
		Notes & Additional Information	3 (0 resolved, 2 partially resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin conducted a diff audit of the [OpenZeppelin/contracts-sui](#) repository, covering the changes introduced by [pull request #417](#) at commit [85448ce](#).

In scope were the following files:

```
contracts
├─ finance
│   └─ sources
│       ├── vesting_wallet.move
│       └── vesting_wallet_linear.move
```

System Overview

The reviewed changes are the diff introduced by [pull request #417](#) to the vesting wallet of the [openzeppelin_finance](#) package, which was audited in full in a prior engagement. The wallet is a curve-agnostic primitive that locks a single coin type for one beneficiary and pays it out over time according to a schedule supplied by a separate curve module. The architecture, the type-level binding between wallet and curve, and the built-in linear curve are described in the prior report and are structurally unchanged. The pull request migrates the wallet's fund flows from [Coin<C>](#) objects to Sui address balances, a model in which funds are credited to addresses by the network's accumulator machinery and settle through system transactions at consensus-checkpoint boundaries. The [vesting_wallet_linear](#) module changes only mechanically, adopting the new payout signature.

Address-Balance Funding and Payout: Funding and payout previously minted and transferred coin objects; both now operate on raw balances:

- [deposit](#) accepts a [Balance<C>](#) from any caller and joins it into the wallet's holdings under the existing lifetime-total overflow guard.
- [release](#) pays the newly vested portion into the beneficiary's address balance through [balance::send_funds](#), minting no coin object; the linear curve's [release](#) delegates to it and no longer takes a transaction context.
- [receive_and_deposit](#) continues to claim coin objects transferred to the wallet's object address. Each funding path now emits a dedicated event: [Deposited](#), [Swept](#), or [Received](#).

Sweeping and Teardown: A wallet's own object address can now accrue settled funds, so the diff adds a path to claim them and a matching teardown check:

- [sweep_settled](#) reads the wallet's settled balance from the shared [AccumulatorRoot](#) and pulls the full amount into the on-book balance, where it participates in the schedule like any other deposit.
- [destroy_empty](#) now takes a reference to the [AccumulatorRoot](#) and completes only when the wallet holds no balance and no settled funds are reported at its address, directing callers to sweep first.

Security Model and Trust Assumptions

The security model established in the prior engagement carries over: the primitive enforces fund conservation and release accounting, schedule correctness is delegated to the curve module that holds the witness, and funding and release remain permissionless. The diff adds no roles or capabilities; it replaces the payment rail, so the added assumptions concern the address-balance machinery on which the wallet now relies.

The system relies on the following trust assumptions:

- Address balances are maintained by the Sui framework's accumulator machinery: funds sent through `balance::send_funds` are credited to the recipient's address by system settlement transactions, and the wallet inherits the integrity of this machinery.
- A release credits the beneficiary's address balance directly. An account beneficiary needs no claim transaction, while an object-address beneficiary is trusted to withdraw the credited funds through its owning logic.
- Funding stays permissionless across all three paths, and the beneficiary remains a data field fixed at construction rather than a capability; the curve module remains trusted to attest honest vested amounts, as set out in the prior report.

The privileged roles are unchanged from the prior engagement:

- **Curve module (holder of the witness):** mints attestations through `mint_vested_amount` and finalizes teardown through `consume_receipt`. The linear curve exercises this authority through `vested_amount` and `destroy`. The latter still requires the schedule to have ended and the caller to be the beneficiary.
- **Beneficiary:** the fixed recipient of every `release`, now credited as an address balance rather than a coin object, and the only finalizer of the linear teardown.
- **Wallet holder (owned topology):** the sole party able to invoke the wallet's mutating entries (funding, sweeping, release, and teardown), since each needs the wallet by reference or value; outside parties can still place funds at the wallet's address for the holder to claim or sweep.

Low Severity

L-01 In-Flight Deposits Can Block `destroy_empty` or Be Permanently Stranded

The `destroy_empty` function tears down a drained `VestingWallet` and reclaims its storage rebate. Besides requiring an empty balance, it aborts with `EUnsweptFunds` when any settled funds reside at the wallet's object address, directing callers to `sweep_settled` first. Settled funds accrue through `balance::send_funds`, which is permissionless: any account can send any amount of the vested coin type to any address, and the wallet [documents this as a supported funding path](#).

Because this check reads state that anyone can modify, a teardown transaction can be forced to abort by dust settled at the wallet's address in an earlier consensus checkpoint (`settled_funds_value` reads a snapshot taken at the start of the current checkpoint). This can happen coincidentally, when an upstream emitter is still streaming into the wallet, or deliberately, since a drained wallet whose schedule has ended is a predictable target and recurring dust keeps the check failing. Recovery requires a sweep followed by a release, which is [possible in full only once the schedule has ended](#) for the linear curve. Since the snapshot only advances when the [system settlement transaction](#) runs at the checkpoint boundary, a retried teardown succeeds only in a later checkpoint. No funds are at risk, and the settled dust is recoverable to the beneficiary through the same sweep and release.

The same snapshot also affects a deposit still in flight: a `Balance<C>` sent in the same checkpoint as the teardown is invisible to the gate, so the check passes, `finish_destroy` deletes the wallet's `UID`, and the funds settle at the boundary against an address that no withdrawal can ever reach again. The deposit is permanently lost, and the [existing caveat](#) warns only about funds sent after the call. For the linear curve, the loss stays with the beneficiary, who [alone can finalize teardown once the schedule has ended](#) and to whom the deposit would have vested; curves without such gates extend the loss to third-party emitters.

Consider strengthening the `destroy_empty` documentation to state that deposits sent in the same consensus checkpoint cannot be detected and are stranded if the teardown proceeds, so integrators must halt upstream emissions and let at least one full checkpoint elapse before tearing a wallet down. Furthermore, consider documenting that a teardown attempt can be made to abort at any time by newly settled third-party funds, so destruction is best treated as

retryable rather than guaranteed. Finally, consider extending the [rationale paragraph](#) of the linear `destroy` to cover this settled residual alongside the unclaimed-coin residual it already mentions.

Update: Resolved in [pull request #463](#). The team stated:

| *Updated documentation*

Notes & Additional Information

N-01 Incomplete `sweep_settled` Abort Documentation

The [style guide](#) requires every `#### Aborts` section to list each cause a function can raise, including native aborts and those propagated from cross-module calls. The [sweep_settled section](#) lists only `EBalanceOverflow`, but the function can also abort with `EObjectFundsWithdrawNotEnabled`, raised inside [withdraw_from_object](#) when the [enable_object_funds_withdraw](#) protocol feature is disabled. In addition, it can fail with the `InsufficientFundsForWithdraw` execution status, raised at [redeem_funds](#) when a withdrawal exceeds the funds actually available, for example on a second sweep in the same consensus checkpoint re-reading the stale settled snapshot.

Consider completing the `#### Aborts` section of `sweep_settled` with the two missing causes.

Update: Partially Resolved in [pull request #464](#). The team stated:

| *Already addressed on `release-v1.4`: the docs already listed `sui::funds_accumulator::EObjectFundsWithdrawNotEnabled` when object funds withdrawal is disabled.*

| *What we fixed: the remaining vague redemption abort note.*

N-02 Misleading Documentation

Several doc comments in `vesting_wallet.move` do not match the code they describe:

- The `sweep_settled_summary` reads "Sweep `amount` from the wallet's own object address balance", but the `signature` has no `amount` parameter; the function always sweeps the entire settled balance, deriving the amount internally.
- The `Received_event documentation` describes the no-op condition as a "deposit" of a zero-value coin, while the emitting `receive_and_deposit` consistently describes itself as a claim; the wording blurs the intended distinction between the `Received` and `Deposited` events.
- The `Swept_event documentation` says a sweep of a "zero-value balance" emits no event, but `sweep_settled` takes no balance argument; the actual `no-op condition` is a wallet with no settled funds at its address.

Consider rewording the three comments to match the implemented behavior and terminology used by the emitting functions and events.

Update: Partially Resolved in [pull request #465](#). The team stated:

Already fixed on `release-v1.4`: `sweep_settled` no longer documents a nonexistent `amount` parameter.

Fixed now: the remaining event docs in `vesting_wallet.move`:

- `Swept` now says it is emitted when settled funds are swept, not when a "zero-value balance" is passed.
- `Received` now uses claim terminology for `receive_and_deposit`, instead of generic deposit/"receival" wording.

N-03 Open TODO in the Code

The `destroy_empty_for_testing` helper added by the pull request bypasses the settled-funds gate so that unit tests can exercise teardown, because the pinned Sui release ships no `AccumulatorRoot` test constructor. Its documentation carries a [TODO comment](#) marking the helper for removal once `accumulator::create_for_testing` ships in the published framework, at which point the teardown tests can be rerouted through the real `destroy_empty`. The helper is test-only, so neither it nor the comment reaches published bytecode, but pending work tracked only through a source comment is easy to lose in a refactor and clutters the code.

Consider removing the TODO from the source and tracking the rework in the project's issue tracker instead, so that the teardown tests are rerouted through the real `destroy_empty` once `accumulator::create_for_testing` ships.

Update: Acknowledged, not resolved. The team stated:

For now I think is easier for us to keep the TODO as we won't be actively maintaining the repo so an issue may be missed, and the TODO, even when not ideal, is easier to find.

Conclusion

This diff audit covered pull request #417 to the vesting wallet of the `openzeppelin_finance` package, whose full audit was the subject of a prior engagement. The pull request migrates the wallet's fund flows from `Coin<C>` objects to Sui address balances: deposits accept raw balances, releases credit the beneficiary's address balance directly, a wallet can sweep funds settled at its own object address into its balance, and teardown refuses to proceed while unswept funds remain. The review assessed the preservation of the wallet's conservation and release-accounting invariants across the new payment rail and the behavior of the new paths against the accumulator and settlement machinery of the Sui framework.

No issues of critical, high, or medium severity were identified within the reviewed changes. One low-severity issue and three notes were raised. The low-severity issue describes how the new teardown gate interacts with funds that are in flight when a wallet is destroyed: amounts settled in earlier consensus checkpoints make the teardown abort, while amounts settling in the same checkpoint cannot be detected and are permanently stranded if the teardown proceeds. The notes address the completeness of the abort documentation on the new sweep path (including its dependency on a protocol feature), several doc comments that the migration left inaccurate, and an open TODO in the source code that tracks pending test rework. The core guarantees, namely fund conservation, the lifetime-total overflow guard, monotonic release accounting, the wallet-ID binding of attestations, fixed-beneficiary payouts, and the witness-gated teardown, were preserved across the migration.

The changes are limited to the payment-rail migration and keep the module's established structure: the linear curve adopts the new payout path without behavioral change, and each funding path reports through a dedicated event. The settlement-dependent paths were assessed against the Sui framework sources, as the corresponding unit tests await an accumulator test constructor in a future framework release; exercising these paths end to end once that constructor becomes available would complement this review.

The OpenZeppelin development team is thanked for their responsiveness and willingness to provide technical context throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.